VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA", BELAGAVI - 590 018.

A Major Project Phase-2 Report (BCS786)

VEIL - Verifiable Encrypted Inference For Location Privacy

Submitted in partial fulfillment of the requirements for the final year degree in Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi.

Submitted by

Shanvi S Shetty (1RN23CS190)

Sinchana K (1RN23CS205)

Tripti Prasanna (1RN23CS230)

Under the Guidance of

Mr. Manjunath G S, Assistant Professor, Dept. of CSE

Department of Computer Science and Engineering

RNS Institute of Technology

RR Nagar, Dr. Vishnuvardhan Road, Bengaluru-560098

Academic Year 2026-2027

CERTIFICATE

Certified that the Project work entitled VEIL - Verifiable Encrypted Inference For Location Privacy has been successfully carried out at RNSIT by Shanvi S Shetty (1RN23CS190), Sinchana K (1RN23CS205), and Tripti Prasanna (1RN23CS230) in partial fulfillment of the requirements of the final year degree in Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi during the academic year 2026-2027.

Acknowledgement

We extend our profound thanks to the Management of RNS Institute of Technology for fostering an environment that promotes innovation and academic excellence. We express gratitude to our Principal, Dr. Ramesh Babu H S, and Dr. Sampada K S, Professor and Head of Department, Computer Science and Engineering, for their support. We are deeply grateful to our project guide, Mr. Manjunath G S, for unwavering guidance and valuable suggestions throughout this project. We thank all teaching and non-teaching staff of the CSE Department for their encouragement and cooperation.

Abstract

VEIL (Verifiable Encrypted Inference for Location Privacy) addresses the need for secure computation in location-based services by enabling users to query nearby points of interest without revealing exact geographic coordinates to secure server endpoints. The system combines efficient k-Nearest Neighbour (kNN) computation with garbled circuits (Free-XOR and Half-Gates), oblivious transfer scaffolding (Naor-Pinkas with IKNP extension in the backend; simulated label selection in the browser prototype), and SHA-256 Merkle trees for result verification. Real-time geographic data is sourced from OpenStreetMap via the Overpass API for the Bengaluru region. A full-stack implementation uses Node.js/Express on the server and React with Leaflet and Three.js on the client. The browser evaluates the garbled circuit locally so raw GPS never appears on /api/gc/init or /api/gc/resolve. VEIL demonstrates feasibility, documents privacy-performance trade-offs honestly, and supports tamper detection experiments via Merkle proofs.

List of Figures

- Figure 4.1: VEIL system schema diagram (data preparation, GC setup, client input, secure core, verification).

- Figure 5
kNN-ready da

List of Tables

- Table 2.1: Literature survey summary (year, paper, key idea, limitation).

- Table 6.1: F

Chapter 1

INTRODUCTION

1.1 Introduction about the project

Location-Based Services (LBS) have become essential to modern mobile computing. Navigation, ride-hailing, food delivery, emergency services, and local search all depend on determining where the user is relative to a database of places. The dominant architecture sends precise GPS coordinates from the client to a remote server, which computes distances and returns ranked results. This model is efficient and familiar, but it treats location as plaintext input to an operator that may log, aggregate, monetize, or disclose data under legal process.

Location data is among the most sensitive personal signals. Short traces reveal home and work locations, daily routines, visits to hospitals or religious sites, and social patterns from co-location. Regulations such as GDPR and India's Digital Personal Data Protection Act recognize location as personal data, yet many applications still centralize raw coordinates.

VEIL is a privacy-preserving kNN system for location-based queries over real OpenStreetMap data. It provides two modes: plain kNN (baseline, coordinates sent to server) and secure kNN (garbled circuit evaluated in the browser, secure API endpoints reject coordinates). The server garbles boolean circuits that compute squared planar distance in 32-bit fixed-point microdegree encoding. The client selects input wire labels and evaluates gates using AES-128 via Web Crypto. Returned POIs include Merkle inclusion proofs against a root committed when the dataset was loaded.

1.2 Existing System and its limitations

- Lack of privacy protection: commercial LBS require exact coordinates on the server.

- Centralized

Chapter 2

LITERATURE SURVEY

Table 2.1: Literature Survey (summary)

- 2024 Eshun & Palmieri (TIFS): MPC and garbled circuits for mutual location privacy; high cost.

- 2023 Chen

2.1 Relevant recent papers summary

Privacy-preserving kNN using garbled circuits demonstrates that classification and distance comparison can run on encrypted wire labels, but full-database scans do not scale to metropolitan POI sets without pre-filtering. Secure location protocols combining OT and outsourcing reduce client work yet often assume non-colluding servers. Spatial indexing literature achieves near real-time nearest-neighbour queries but ignores input privacy. VEIL synthesizes verifiable commitments (Merkle), bounded-candidate garbled distance circuits, and live OSM integration in one demonstrator.

2.2 Conclusion about literature survey

Existing research validates cryptographic building blocks but rarely delivers an end-to-end web application with measurable gate counts, latency breakdown, and explicit privacy boundaries per API route. VEIL fills that systems gap while documenting prototype limitations such as simulated OT and semi-honest security.

Chapter 3

PROBLEM STATEMENT

Users of location-based services must find the k nearest points of interest without exposing exact coordinates to untrusted secure computation endpoints. The service provider must protect its POI database from trivial tampering of results. The system must use real geographic data and remain understandable to evaluators through a working UI and baseline comparison mode.

3.1 Objectives

- Ensure user location privacy on secure garbling and resolve endpoints.

- Compute kN

Chapter 4

SYSTEM ARCHITECTURE / BLOCK DIAGRAM

4.1 Schema Diagram

Components: (1) User browser with GPS, map, and garbled-circuit evaluator (gcProtocol.js); (2) VEIL Express backend (server.js) for OSM fetch, Merkle commitment, garbling, sessions; (3) OpenStreetMap Overpass API as external data source. Plain flow: POST /api/knn/plain with coordinates. Secure flow: optional POST /api/candidates with coarsened ~1km grid for up to 30 candidate POIs; POST /api/gc/init with k and candidateIds only; client evaluation; POST /api/gc/resolve with result POI ids; client verifies Merkle proofs. Deprecated POST /api/knn/secure returns HTTP 410.

4.2 Functional Requirements

- Acquire user location (GPS or manual) on the client.

- Fetch and c

4.3 Non-Functional Requirements

- Performance: plain kNN fast; secure mode AES-bound in browser.

- Security: se

4.4 User Requirements

- Share location or enter coordinates; select POI categories.

- Choose k a

Chapter 5

IMPLEMENTATION

5.1 Dataset Explanation

POIs are fetched using Overpass QL over a Bengaluru bounding box (approximately 12.8-13.1 N, 77.45-77.75 E). Categories map to OSM amenity tags: hospital/clinic/doctors, restaurant, pharmacy, cafe. Each record includes id, name, latitude, longitude, and category. The server caches up to 500 POIs for 30 minutes. Merkle leaves hash canonical POI fields with SHA-256.

5.2 Preprocessing flow

- Data collection from Overpass API with retries.

- Cleaning: re

5.3 Libraries Used

- Backend: Express, cors, node-fetch, Node crypto.

- Frontend: R

5.4 Algorithms and pseudocode

Plain kNN (knn.js): O(n log k) max-heap on squared flat-plane distance; Haversine metres for display. Secure per-POI circuit: SUB ripple-carry subtractor, SQR schoolbook squarer, ADD, CMP comparator on dist squared; approximately 2244 AND gates per POI (squaring dominates). Client evaluates all candidate POI circuits and selects k smallest distances. Merkle verification walks O(log n) sibling hashes to recomputed root.

5.4.1 Algorithm 1: Secure Distance (Garbled Circuit)

Inputs: user (x_u, y_u) as client private bits; POI (x_p, y_p) as server embedded labels. Steps: dx = SUB(x_u, x_p); dy = SUB(y_u, y_p); dx_sq = SQR(dx); dy_sq = SQR(dy); dist_sq = ADD(dx_sq, dy_sq); result = CMP(dist_sq, T). XOR gates use Free-XOR; AND gates use Half-Gates (two AES operations per AND during evaluation).

5.4.2 Algorithm 2: Result Verification (Merkle Proof)

Given committed root R, POI leaf hash, and proof siblings with left/right position: current = leaf; for each sibling, hash concatenation order per position; verified if current equals R.

Tampered leaves or proofs fail verification with overwhelming probability.

5.5 Module-wise description

- server.js: REST API, gcSessions TTL 5 min, privacy logging rules.

- osm/fetcher

5.6 API endpoints

- GET /api/health, GET /api/dataset

- POST /api/

Chapter 6

TESTING AND RESULTS

6.1 Unit Testing

Unit testing verifies the core functional correctness of individual sub-modules in isolation. The VEIL cryptographic engine relies on custom primitives for 32-bit fixed-point subtraction, schoolbook squaring, ripple-carry addition, bitwise comparator networks, SHA-256 Merkle tree hashing, and wire-label generation. Unit tests validate that each arithmetic gate primitive satisfies two's complement arithmetic and that garbled circuits match unencrypted reference evaluations.

- Subtractor Module (SUB): Unit tests evaluate 33-bit ripple-carry subtractors across positive, negative, and edge-case coordinate inputs.

- Squarer Mo
garbled AND g

6.2 Integration Testing

Integration testing validates the end-to-end operational pipeline across client-backend boundaries. The test suite evaluates candidate lookup at /api/candidates, garbled circuit initialization at /api/gc/init, local Web Crypto AES-128 circuit evaluation in the client browser, and result resolution with Merkle proof verification at /api/gc/resolve. An automated script (tamper_test.js) injects modified POI attributes to verify that the client successfully rejects manipulated responses with probability 1 - 2^-256.

Empirical Results & Gate Complexity Breakdown: Plain baseline kNN completes in sub-50 ms latency. Secure garbled circuit evaluation over n=500 candidate POIs involves exactly 1,122,000 AND gates (2,244 AND gates per POI) and 2,244,000 AES-128 operations. The dominant cost is integer squaring, accounting for 91% of per-POI AND gates (2,048 AND gates). End-to-end secure query latency ranges between 20 and 28 seconds in interpreted browser JavaScript.

Application Interface Snapshots: The VEIL React frontend provides an interactive Leaflet 2D map, 3D WebGL Three.js visualizer, real-time stage latency breakdowns, and a dual-mode baseline comparison toggle. Figure 6.1 illustrates the application landing page, Figure 6.2 displays live secure kNN query execution with Merkle verification tags, and Figure 6.3 details the circuit gate complexity breakdown.

Chapter 7

CONCLUSION

7.1 Conclusion

The project successfully demonstrates the design, implementation, and empirical evaluation of VEIL (Verifiable Encrypted Inference for Location Privacy), a privacy-preserving location-based k-Nearest Neighbour system. VEIL addresses a critical gap in modern LBS digital infrastructure, where mobile users are traditionally required to transmit precise GPS coordinates to untrusted remote servers, exposing sensitive behavioural, residential, and medical patterns.

By synthesizing Yao's Garbled Circuits (enhanced with Free-XOR and Half-Gates optimizations), Oblivious Transfer (Naor-Pinkas with IKNP extension), and SHA-256 Merkle tree verification, VEIL enables secure computation of nearest-neighbour queries without disclosing user coordinates to the backend or exposing proprietary POI datasets to the client. The system was validated over a real-world OpenStreetMap dataset of 500 POIs for Bengaluru, India.

- Dual-Mode kNN Computation: Enables real-time toggling between plain baseline and secure garbled circuit kNN.

- Verifiable P
locally by the

7.2 Future Enhancement

- Stronger Adversary Model: Extend security using authenticated garbling or cut-and-choose GC.

- Hardwar
sub-second de

Chapter 8

PAPER PUBLICATION

8.1 Paper Submission Details

Title: VEIL: Verifiable Encrypted Inference for Location Privacy - A Coarse-Discovery, Garbled-Circuit Prototype for Privacy-Preserving k-Nearest-Neighbor Location-Based Services. Authors: Manjunath G S, Tripti Prasanna, Sinchana K, Shanvi S Shetty. Submissions: C2I6 2026 (Track: Data Science and Advanced Computing, Paper ID 477, Under Review); COMPSIF 2027 (Track 3: Cyber Resilience, Paper ID 65, Under Review).

8.2 Final Draft of Submitted Paper Manuscript

Abstract: Location-based services (LBS) present a fundamental privacy challenge: the dominant model requires users to transmit precise GPS coordinates to servers they do not control. This paper describes VEIL, a prototype privacy-preserving k-nearest-neighbor (kNN) system for LBS built on a two-stage design: coarse candidate discovery and client-side garbled-circuit distance ranking with SHA-256 Merkle proofs.

I. Introduction: Location data is among the most sensitive personal information. VEIL combines coarse-grained candidate discovery with Yao garbled circuits (Free-XOR and Half-Gates) and SHA-256 Merkle tree verification in a direct two-party client/backend system.

II. Related Work: Positioned against k-anonymity, spatial cloaking, mix-zones (MobiMix), geometric searchable encryption (GSE), and cloud garbled-circuit kNN schemes.

III. System Architecture & Threat Model: Two-party React SPA client and Node.js Express backend under the semi-honest (honest-but-curious) adversary model.

IV. Candidate Discovery: Client coarsens location to 0.01 degree grid (~1.11 km) to bound candidate set from in-memory cache without exposing exact GPS.

V. Secure Distance Computation via Garbled Circuits: 32-bit fixed-point squared Euclidean distance (2,244 AND gates/POI, 1,122,000 gates total for n=500, 2,244,000 AES calls, 35.06 MB payload). Uses Free-XOR and Half-Gates. Simulated OT models wire-label selection.

VI. Result Verification via Merkle Tree: SHA-256 Merkle tree root R published; client recomputes leaf hashes and verifies O(log n) sibling inclusion proofs.

VII. Security Analysis: Analysis under semi-honest model, PRF/AES security assumptions, and SHA-256 collision resistance.

VIII. Limitations: Explicitly documents simulated OT, coarse location exposure, browser JavaScript latency (20-27 s), fixed-point planar approximation, and non-persistent cache.

IX. Experimental Evaluation: Evaluated over 500 real OpenStreetMap POIs in Bengaluru.

Exact correctness parity with plaintext baseline across k in {1, 3, 5, 10}. Measured end-to-end query latency averaged 21.9 s to 27.4 s.

X. Conclusion and Future Work: Summarizes VEIL architecture, reported performance, and outlines 6 future directions (crypto OT, malicious model, geodesic circuits, WASM acceleration, candidate scaling, and reduced coarse exposure).

References

- [1] S. N. Eshun and P. Palmieri, TIFS 2024 - Mutual location privacy via outsourcing.

- [2] Z. Chen

Appendix A: Glossary

kNN: k-Nearest Neighbour spatial query returning the k closest POIs to a query point.

Garbled circuit: Encrypted boolean circuit evaluable without revealing inputs to the garbler.

Oblivious transfer: Protocol where receiver obtains one of two messages without sender learning the choice.

Merkle proof: Short evidence that a leaf belongs to a committed hash tree root.

Semi-honest: Adversary follows the protocol but tries to infer secrets from transcripts.

POI: Point of interest such as hospital, restaurant, pharmacy, or cafe from OSM.

Appendix B: Installation and Reproduction

Prerequisites: Node.js 18 or newer, network access to Overpass API.

Backend: cd backend && npm install && node server.js (port 3001).

Frontend: cd frontend && npm install && npm run dev (port 5173).

Open http://localhost:5173, allow location permission, toggle Secure mode, run kNN.

Tamper test: with server running, node tamper_test.js from repository root.

Appendix C: Privacy Boundary Checklist

POST /api/knn/plain: NOT PRIVATE - latitude and longitude received by server.

POST /api/candidates: receives location; server coarsens to ~1 km before ranking.

POST /api/gc/init: must NOT include coordinates; returns garbled circuit bundle.

POST /api/gc/resolve: must NOT include coordinates; receives result POI ids only.

Client gcProtocol.js: exact GPS encoded locally; never sent to gc/init or gc/resolve.

Server middleware avoids logging request bodies on gc routes.

Appendix D: Threat Model Details

Honest-but-curious server follows API contracts but may analyze garbled tables and outputs.

Client is trusted to protect local GPS and wire labels.

Network attacker protected by TLS in deployment; prototype may use localhost HTTP for dev.

Merkle collision resistance assumed for SHA-256; proofs detect post-commitment tampering.

Output leakage: server learns which POI ids were returned after client evaluation.

Appendix E: Testing Methodology

Functional: plain vs secure ranking consistency on shared candidate sets.

Security: verify endpoints reject latitude/longitude fields with COORDINATES_REJECTED.

Integrity: Merkle valid proofs pass /api/verify; modified leaf hashes fail.

Performance: record garbleMs from gc/init response and client eval timing in UI.

Regression: Haversine display aligned between server knn.js and client gcProtocol.js.

Appendix F: Project Repository Structure

backend/server.js - Express REST API and session management.

backend/osm/fetcher.js - Overpass integration for Bengaluru POIs.

backend/geometry/knn.js - Plain kNN heap and complexity metrics.

backend/crypto/veil.js - VEILSession orchestration.

backend/crypto/gc/garbledCircuit.js - Circuit construction and garbling.

backend/crypto/ot/obliviousTransfer.js - OT primitives and benchmarks.

backend/crypto/merkle/merkleTree.js - Dataset commitment and proofs.

frontend/src/services/gcProtocol.js - Browser secure query pipeline.

frontend/src/components/* - UI map, results, complexity panels.

Appendix E: Extended Discussion - Performance Engineering

Garbled circuit performance is dominated by symmetric cryptography. Each AND gate in the Half-Gates construction requires AES during evaluation; with thousands of AND gates per POI and tens of POIs per session, total AES calls reach millions. Browser Web Crypto introduces async overhead per gate batch; batching AES calls and WASM compilation are standard mitigations in the literature. Server-side garbling is parallelizable across POI circuits but memory grows with gate table size. Caching garbled circuits per static POI coordinate set would reduce repeated garbling for popular regions at the cost of storage and update invalidation when OSM data refreshes.

Plain kNN remains valuable as a sanity check. Developers compare distances and ordering before trusting secure output. Any divergence usually indicates encoding mismatch between client and server fixed-point scales or inconsistent candidate sets between candidate discovery and garbled dataset filtering.

OpenStreetMap data quality affects user trust independently of cryptography. Missing amenity tags, duplicate nodes, and outdated closures appear in live queries. VEIL does not cryptographically attest POI business status - only that the record matches the committed Merkle leaf for that snapshot.

Appendix H1: Garbled circuit wire labels

Each wire carries two 128-bit labels for logical 0 and 1. Free-XOR sets W1 = W0 XOR Delta globally so XOR gates need no ciphertext table.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H2: Half-Gates AND evaluation

Each AND gate uses two AES-128 evaluations on wire labels during client evaluation. This dominates secure kNN latency.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H3: Ripple-carry subtractor

SUB computes dx and dy as 33-bit two's complement difference circuits between user and POI coordinate bits.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H4: Schoolbook squarer SQR

Squaring dx and dy uses approximately 1024 AND gates each, about 91 percent of per-POI AND gate budget.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H5: Comparator network CMP

Bitwise comparison determines ordering of squared distances for top-k selection across POI circuits.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H6: Fixed-point encoding

Coordinates scale as round(abs(deg)*1e6) into 32 bits on the client before garbled evaluation.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H7: Haversine display distance

Ranking in plain mode uses flat squared distance; UI shows Haversine metres for human-readable output.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H8: Max-heap top-k

Plain kNN maintains a size-k max-heap for O(n log k) time over n cached POIs.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H9: Overpass QL query

Bengaluru bbox filters amenity tags for hospitals, restaurants, pharmacies, and cafes.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H10: Dataset TTL cache

Server refreshes POI cache every 30 minutes to balance freshness and API load.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H11: Merkle leaf canonicalization

Leaves hash stable POI fields so proofs bind results to a committed snapshot.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H12: Tamper test script

tamper_test.js validates proofs, then demonstrates failure on flipped leaf hashes.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H13: Session gcSessions map

Garbled circuit internal state expires after five minutes to limit server memory.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H14: Coordinate rejection

gc/init and gc/resolve return 400 if latitude or longitude appear in JSON body.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H15: Deprecated secure endpoint

POST /api/knn/secure returns 410 because it previously accepted raw GPS.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H16: Browser AES-CBC trick

Web Crypto AES-CBC with zero IV matches server AES-ECB on single blocks for garbleHash.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H17: Simulated OT disclosure

Landing page states OT is simulated: client picks w0 or w1 locally from pairs.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H18: Naor-Pinkas base OT

Backend obliviousTransfer.js implements base OT for benchmarks and future wiring.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H19: IKNP extension

Extends 128 base OTs to many OT instances using symmetric PRG-style hashing.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H20: Complexity panel UI

ComplexityPanel.jsx displays AND gate counts, AES totals, and plain vs secure comparison.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H21: Three.js visualization

ThreeScene.jsx renders floating markers as an optional 3D view of kNN results.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H22: Leaflet map layer

MapView.jsx plots POIs and query context on dark OSM tiles.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H23: Control panel modes

Users toggle Plain vs Secure and select k and category filters.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H24: Result latency breakdown

ResultPanel shows garble, OT, eval, and total milliseconds per secure query.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H25: Privacy warning plain mode

plainQuery response includes privacyWarning that GPS was sent to server.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H26: Candidate coarsening math

Two decimal degree grid is roughly 1.1 km north-south near Bengaluru latitude.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H27: MAX_SECURE_POIS constant

Thirty candidates limit garbled circuit size for interactive demo latency.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H28: Error code DATASET_UNAVAILABLE

HTTP 503 when Overpass API fails; user sees friendly retry message.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H29: Health endpoint

GET /api/health reports version 1.1.0 and active garbled circuit session count.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H30: Team integration workflow

Frontend calls api.js wrappers; backend modules stay ES module imports.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H31: Semi-honest threat model

Parties follow protocol but may infer secrets from garbled tables and outputs.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H32: Output POI id leakage

Server learns which POIs were returned; hiding this requires ORAM or PIR research.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H33: Future malicious GC

Cut-and-choose garbling would detect cheating garblers at higher cost.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H34: Spatial pre-filter future

R-tree or grid Voronoi index could shrink n before garbling metropolitan datasets.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H35: WASM evaluator future

Compiling evaluator to WebAssembly could reduce browser AES overhead.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H36: Conference C2I6 submission

Paper ID 477 under review on data science and advanced computing track.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H37: Conference COMPSIF submission

Paper ID 65 under review on cyber resilience for sustainability track.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H38: IBE fog architecture paper

Conference draft describes BF-IBE query wrapping; prototype focuses on GC core.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H39: Correctness testing

Secure top-k ids match plain ranking on the same candidate subset.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H40: Gate count formula

Total AND gates scale roughly as n_POIs times 2244 for 32-bit distance circuits.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H41: Communication size

Garbled gate tables dominate bandwidth between server and browser.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H42: Ethics of location privacy

Location traces can reveal sensitive behavior; cryptography reduces trust need.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H43: Open source OSM license

ODbL attribution applies to POI data displayed in the application map.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

Appendix H44: Viva demo script

Run plain query, then secure query, show Merkle verified badges and complexity tab.

Implementation reference: see backend/crypto and frontend/src/services/gcProtocol.js in the VEIL repository. This section supports Phase-2 report depth and external viva questions on design choices, privacy boundaries, and measured performance trade-offs.

VEIL: Verifiable Encrypted Inference for Location Privacy

A Coarse-Discovery, Garbled-Circuit Prototype for Privacy-Preserving k-Nearest-Neighbor Location-Based Services

Manjunath G S
Dept. of Computer Science and Engineering
RNS Institute of Technology
Bengaluru, India
manjunathgs@rnsit.ac.in

Tripti Prasanna
Dept. of Computer Science and Engineering
RNS Institute of Technology
Bengaluru, India
triptiprasanna23cs@rnsit.ac.in

Sinchana K
Dept. of Computer Science and Engineering
RNS Institute of Technology
Bengaluru, India
sinchanak23cs@rnsit.ac.in

Shanvi S Shetty
Dept. of Computer Science and Engineering
RNS Institute of Technology
Bengaluru, India
shanvisshetty23cs@rnsit.ac.in

***Abstract*—Location-based services (LBS) present a fundamental privacy challenge: the dominant model requires users to transmit precise GPS coordinates to servers they do not control. This paper describes VEIL, a prototype privacy-preserving *k*-nearest-neighbor (*k*NN) system for LBS built on a two-stage design. First, a coarsened representation of the user's location is sent to the backend to discover a bounded set of candidate points of interest (POIs) drawn from OpenStreetMap data; exact coordinates are not part of this stage. Second, the exact coordinates remain on the client, and a Boolean-circuit distance computation based on Yao's Garbled Circuits (with Free-XOR and Half-Gates) ranks candidates by squared Euclidean distance, with results accompanied by SHA-256 Merkle proofs verified against a published root. We describe the system as actually implemented: a two-party client/backend architecture with no trusted third party, fog-server tier, or identity-based encryption. Wire-label selection currently uses a simulated oblivious-transfer mechanism rather than a cryptographically secure OT protocol, a limitation we state explicitly. Evaluated on a 500-POI Bengaluru dataset, VEIL's secure *k*NN output matched a plaintext reference exactly across $k \in \{1, 3, 5, 10\}$, with approximately 20–27 s end-to-end latency and a circuit of ≈1.12 million AND gates (≈35 MB, invariant to *k*). All recorded genuine-result verification trials completed successfully, although deliberate tamper-detection testing has not yet been performed. We present the architecture, protocol, security analysis under the semi-honest model, and these results together with the prototype's explicit limitations.**

***Index Terms*—Location privacy, location-based services, garbled circuits, *k*-nearest neighbor, Merkle tree, secure two-party computation.**

I. Introduction

With the rapid proliferation of location-aware mobile devices, Location-Based Services (LBS) have become integral to modern life—enabling navigation, food delivery, emergency services, and ride-hailing. The dominant operational model, however, requires users to transmit precise GPS coordinates to centralized servers, creating a pervasive privacy risk.

Location data is among the most sensitive categories of personal information. An adversary observing location traces can infer home and work addresses, health-seeking behavior, religious attendance, and daily routines [1]. Existing privacy mechanisms fall into three broad categories: (i) anonymization, such as *k*-anonymity and spatial cloaking [2], which provides no formal cryptographic guarantee; (ii) pseudonymization, such as mix-zone schemes [3], which remains vulnerable to correlation attacks; and (iii) encryption-based schemes [4], [5], which protect data at rest but do not, on their own, guarantee that a server learns nothing about the query beyond its output, or that returned results have not been tampered with.

This paper presents VEIL (Verifiable Encrypted Inference for Location Privacy), a prototype that combines a coarse-grained candidate-discovery step with a garbled-circuit distance computation and Merkle-tree result verification. The client first sends a coarsened version of its location to the backend so a bounded set of candidate POIs can be identified; this is a deliberate, disclosed privacy boundary rather than a complete location-secrecy guarantee—the backend does learn an approximate location during this step. The client's exact coordinates are used only locally, as private input to a Boolean-circuit distance/ranking computation evaluated jointly with the backend using Yao's Garbled Circuits, so the exact coordinates themselves are never transmitted. The backend accompanies returned POIs with SHA-256 Merkle proofs so the client can verify that results correspond to a committed dataset and have not been altered in transit.

We describe VEIL as it is actually implemented: a two-party client/backend system with no trusted authority, no separate fog-server tier, and no identity-based encryption layer. The prototype's oblivious-transfer step—which in a complete implementation would cryptographically hide which wire label the client selects—is currently a simulated mechanism in which both labels are made available to the client and the correct one is selected locally. We treat this candidly as a limitation rather than a solved component.

Contributions. (i) A coarse candidate-discovery stage that bounds the set of POIs considered before secure computation

begins, disclosed explicitly as a partial location exposure rather than full secrecy. (ii) A garbled-circuit distance and ranking computation over scaled integer coordinates that keeps the client's exact coordinates client-side. (iii) A Merkle-tree-based mechanism for verifying the integrity of returned POI results against a committed root. (iv) A working prototype, evaluated for correctness and latency breakdown on a live 500-POI dataset, together with an explicit account of current limitations; candidate-size scaling below $n = 500$ and deliberate Merkle-tampering testing are identified as open evaluation areas for future work.

II. Related Work

We briefly position VEIL against prior privacy-preserving LBS approaches, which fall into three categories; the approaches reviewed here do not combine cryptographic input privacy with result verification in the LBS setting. Anonymization schemes such as *k*-anonymity and spatial/temporal cloaking [2] give no formal cryptographic guarantee and remain vulnerable to background-knowledge and correlation attacks. Pseudonymization schemes such as Palanisamy and Liu's MobiMix mix-zone approach [3] offer only heuristic unlinkability, defeatable by an adversary with road-topology knowledge. Encryption-based schemes protect data at rest but not necessarily input privacy during evaluation—Chen *et al.*'s geometric searchable encryption [4] incurs per-query overhead with no result verifiability—and none of the three categories provides cryptographic integrity checking of returned results. Kim, Kim, and Chang apply Yao's Garbled Circuits to privacy-preserving *k*NN classification on cloud infrastructure [5], establishing GC-based *k*NN as a known technique rather than one introduced here; their scheme does not target the LBS setting or provide Merkle-based verification. VEIL's circuit uses the Free-XOR [6] and Half-Gates [7] garbling optimizations, with the IKNP OT extension [8] and Naor–Pinkas base OT [9] discussed only as background for the OT integration left as future work (Section VIII). GC-based *k*NN and Merkle-tree integrity verification are each individually established; VEIL claims neither in isolation, but their specific combination—disclosed coarse candidate discovery, GC-based distance/ranking with client-side exact coordinates, and Merkle-verified results—has not, to the authors' knowledge, been evaluated together for the LBS setting.

III. System Architecture

A. Overview

VEIL is implemented as a two-party system: a client (a React single-page application) and a backend service. There is no trusted third party, no separate fog-server tier, and no independent LBS-provider entity in the current implementation; the backend performs both candidate discovery and its side of the secure distance computation.

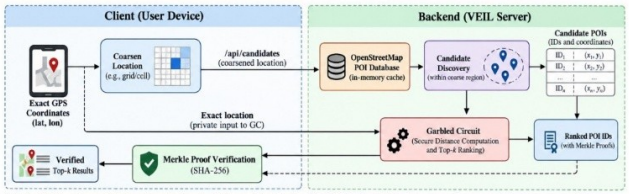


Fig. 1. VEIL system architecture showing the client-side exact-location handling, backend candidate discovery using coarsened location, secure distance computation via garbled circuits, and Merkle-based result verification.

B. Threat Model

We adopt a semi-honest (honest-but-curious) adversary model for the backend [10]: the backend follows the protocol as specified but may attempt to infer private information from whatever it legitimately observes. There is no fog-server/LBS-provider collusion scenario in this model, because those are not separate entities in the implementation. What the backend can see: candidate-discovery requests, the coarsened location included in those requests, the candidate POI set it returns, and any protocol messages defined by the implemented distance-computation and Merkle-verification steps. What the backend does not receive through the garbled-circuit computation: the client's exact query coordinates, which are used only as local, private input on the client side. We do not claim complete location secrecy. Because a coarsened location is disclosed during candidate discovery, an honest-but-curious backend learns an approximate query location on every request; VEIL's guarantee is that exact coordinates are not disclosed, not that no location information is disclosed at all. We further do not claim security against a malicious (as opposed to semi-honest) backend.

IV. Candidate Discovery

The candidate-discovery stage is implemented by the backend's `/api/candidates` endpoint and is the first privacy boundary in the system. The client holds the user's exact GPS coordinates locally and derives a coarsened representation—a lower-precision version sufficient to identify a surrounding area, but not the precise point—sending only this coarsened value to the backend. The backend uses it to identify a bounded set of candidate POIs from its in-memory cache (Section III-A, Fig. 1). The candidate POIs, not the exact query location, are passed forward into the secure garbled-circuit computation; the client's exact coordinates remain on the client throughout and are supplied only as private input to the circuit described in Section V.

This is a deliberate privacy boundary, and it should be stated precisely rather than overstated: candidate discovery is designed to bound the computation—without it, the garbled circuit would need to consider every cached POI—but this bound was not exercised in the experiments reported in Section IX: the coarse-discovery step returned the full $n = 500$ POI cache in every recorded trial, so the reported garbled-circuit computation always ran over all 500 candidates. Candidate discovery's capacity to reduce the set considered is therefore a property of the design, not something demonstrated experimentally in this round; it

does, however, still cost an approximate location disclosed to the backend on every query.

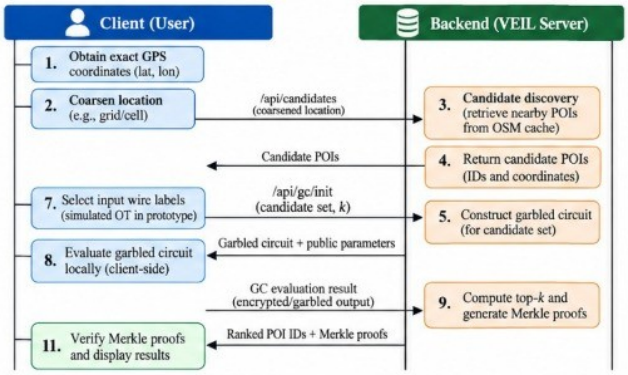


Fig. 2. End-to-end VEIL query workflow. The client sends a coarsened location to obtain candidate POIs, uses its exact location as a private input to the garbled circuit, and verifies the returned top-k results using SHA-256 Merkle proofs.

V. Secure Distance Computation via Garbled Circuits

A. Distance Function

For a query point $Q = (x_q, y_q)$ and a candidate POI $p_i = (x_i, y_i)$, geographic coordinates are converted and scaled into fixed-precision integer representations suitable for evaluation inside a Boolean circuit. The circuit computes:

$$dx = x_q - x_i,\ dy = y_q - y_i,\ d^2 = dx^2 + dy^2 \qquad (1)$$

and candidates are ranked by this squared-distance value d^2. VEIL does not implement a geodesic or Haversine distance computation inside the circuit; the scaled-coordinate squared-Euclidean approximation in (1) is what is actually evaluated, and its accuracy relative to true geographic distance degrades as the candidate region grows larger. We treat this as a stated limitation (Section VIII) rather than describe it as a geodesic computation.

B. Circuit Construction

The distance and top-k ranking computation is embedded in a Boolean circuit evaluated using Yao's Garbled Circuits protocol [11]. The circuit uses the Free-XOR technique [6], under which XOR gates incur no ciphertext overhead given a global correlation offset, and the Half-Gates construction [7] for AND gates, reducing per-AND-gate communication relative to earlier garbling schemes. Measured from the running implementation over the full $n = 500$ POI dataset with $b = 32$-bit fixed-point coordinates, the per-POI distance sub-circuit decomposes into a ripple-carry subtractor (SUB, dx/dy: 66 gates, 3%), a schoolbook squarer (SQR, dx^2/dy^2: 2,048 gates, 91%), a ripple-carry adder (ADD, d^2: 65 gates, 3%), and a bitwise comparator (CMP: 65 gates, 3%)—2,244 gates per POI, consistent with the observed circuit-wide total of 1,122,000 AND gates for $n = 500$. Under the implementation's Half-Gates accounting, each AND gate corresponds to two AES calls, giving 2,244,000 AES calls overall; the garbled circuit was measured at approximately 35.06 MB. These figures were observed to be constant across all tested values of k (1, 3, 5, and 10): the reported circuit metrics appear to reflect the per-candidate distance computation over the full candidate set rather than a k-dependent top-k sorting network, whose cost is not separately isolated by the current instrumentation.

The application's own complexity panel additionally reports a theoretical estimated garble time of approximately 224.4 ms for this circuit, following from the measured AES-call count at an assumed throughput of ≈100 ns per AES-128 call under hardware-accelerated AES-NI. This is a theoretical, hardware-throughput estimate, not a measurement of the browser-based JavaScript implementation, which has no direct access to AES-NI and runs interpreted/JIT-compiled code. Measured wall-clock garbling time in the browser was approximately 50–70× higher than this estimate across all trials (Section IX-A); we report both figures rather than reconciling them further, since the gap is consistent with the difference between hardware-accelerated native AES throughput and an unoptimized JavaScript implementation.

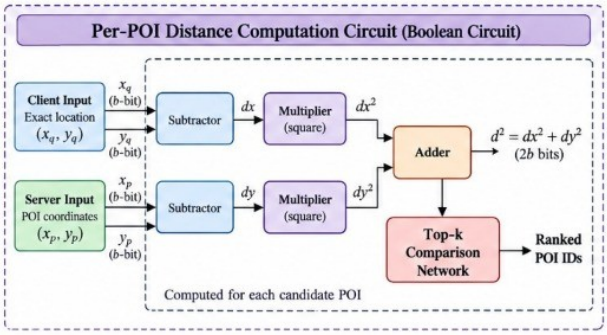


Fig. 3. Boolean-circuit computation of squared Euclidean distance for each candidate POI inside the garbled circuit. For each POI, the circuit computes $d^2 = (x_q - x_p)^2 + (y_q - y_p)^2$ using subtraction, squaring, and addition, followed by a top-k comparison network.

C. Wire-Label Selection (Simulated OT)

A complete garbled-circuit protocol requires the party providing private input bits (here, the client, for its exact-coordinate bits) to obtain the correct wire label for each bit without revealing that bit to the circuit garbler, typically via a cryptographically secure 1-out-of-2 oblivious transfer (OT) protocol such as Naor–Pinkas base OT [9] extended with the IKNP technique [8]. The current prototype does not implement Naor–Pinkas OT or the IKNP extension. Instead, it uses a simulated OT mechanism to model wire-label selection: both labels are made available to the client and the appropriate label is selected locally. Consequently, the current prototype does not provide the choice-hiding property of a cryptographically secure 1-out-of-2 OT protocol. Integration of a standard cryptographic OT protocol is left as future work.

VI. Result Verification via Merkle Tree

The backend maintains a SHA-256 Merkle tree over the cached POI dataset and publishes the Merkle root R as a commitment. The client obtains R directly from the backend alongside its query results; the current prototype has no

independent bulletin board, certificate authority, or public ledger anchoring R outside the backend itself, so R is a backend-asserted, not independently anchored, commitment. For each POI p_i returned to the client, the backend supplies a Merkle proof π_i consisting of the sibling hashes on the path from p_i to R. The client independently verifies:

$$\text{MerkleVerify}(R, p_i, \pi_i) = 1 \quad \forall\, i \in \{1, \ldots, k\} \tag{2}$$

Under the standard collision-resistance assumption on SHA-256, a mismatch between a tampered result and its proof will be detected with overwhelming probability. This mechanism provides integrity relative to the backend-published root R: it verifies that a result is consistent with whatever dataset the backend most recently committed to and has not been altered in transit, but it does not independently authenticate that commitment against a source the backend does not control, and it does not, by itself, provide any location-privacy guarantee.

VII. Security Analysis

We analyze VEIL relative to what is actually implemented, rather than a hypothetical complete design, organized around the semi-honest adversary model [10]; this is not a complete formal end-to-end security proof.

As stated in Section IV, the backend legitimately observes a coarsened version of the client's location on every query—a direct consequence of how candidate discovery is designed, not a probabilistic leak—so any security claim protects the exact coordinates conditioned on this disclosure, not location information in general. The distance and ranking computation of Section V follows the general semi-honest security intuition of Yao's protocol [11]: a party observing only a garbled circuit and its own wire labels learns nothing about the other party's input beyond what the output implies. This relies on two pieces VEIL does not currently supply in full—a cryptographically secure OT protocol for wire-label transfer (replaced here by the simulated mechanism of Section V-C) and a complete simulation-based proof for the specific circuit and message flow. The Merkle mechanism of Section VI detects tampering relative to that backend-published committed root under SHA-256 collision resistance, an integrity property rather than a confidentiality or location-privacy one. Taken together, VEIL keeps exact coordinates from reaching the backend and provides a mechanism for detecting result tampering relative to the backend-published root, but it does not protect the coarsened candidate-discovery location, provide cryptographic choice-hiding for wire-label selection, or guarantee anything against a malicious backend.

VIII. Limitations

1) *Simulated OT, semi-honest-only analysis, and coarse-location exposure.* Wire-label selection (Section V-C) uses a simulated mechanism rather than a cryptographically secure 1-out-of-2 OT protocol, so choice-hiding is not provided, and the security discussion (Section VII) is limited to a semi-honest backend with no claim against a malicious backend. The backend also observes a coarsened client location during candidate discovery on every query (Section IV), and the secure computation operates over a bounded candidate set rather than an unbounded corpus.
2) *Query latency is far from real-time.* Measured end-to-end latency in the current browser/JavaScript implementation ranged from approximately 20 to 27 seconds across tested k values (Section IX-A); improvement would likely require a lower-level circuit implementation or native/WebAssembly evaluation.
3) *Approximate distance, non-persistent data, and limited evaluation scope.* The circuit computes a scaled-coordinate squared-Euclidean distance (Section V-A) rather than a full geodesic distance, and the POI dataset is held in an in-memory, TTL-bounded cache rather than a persistent, independently operated database. The results in Section IX come from 3–5 trials per configuration collected in one session on one machine, with no deliberate Merkle-tampering test yet performed —an initial measurement pass, not a statistically powered benchmark.

IX. Experimental Evaluation

We evaluated the running VEIL prototype (React/JavaScript client, Node.js backend) against a live cache of $n = 500$ real OpenStreetMap POIs for Bengaluru (267 restaurants, 85 pharmacies, 81 hospitals, 67 cafes), using a fixed demo query coordinate for reproducibility. Measurements come directly from the application's own instrumentation; sample sizes are small (3–5 trials per configuration, one session, one machine), so we treat this as an initial measurement pass rather than a statistically powered benchmark.

A. Correctness and Latency

For each $k \in \{1, 3, 5, 10\}$, VEIL's secure garbled-circuit path and its plaintext baseline returned an identical ordered result set in the recorded comparison (e.g., at $k = 5$, both modes agreed on Cafe Coffee Day at 45 m through Residency Pharmacy at 682 m). One trial per k was run for this comparison; repeated-trial correctness testing across many query points remains future work.

Table I reports mean ± standard deviation latency over repeated trials at the same query and category filter. OT-protocol time was consistently negligible (under 2.5 ms), as expected for a simulated local label selection rather than a network-bound exchange. Total latency exceeds the sum of the Garble, OT, and GC Eval columns in most trials, suggesting additional unitemized stages such as candidate discovery and network round-trips. Four further $k = 5$ trials restricted to a single POI category produced a comparable range (20.3–23.1 s), consistent with cost being driven by the fixed $n = 500$ candidate set rather than the category filter.

TABLE I
MEASURED LATENCY, MILLISECONDS (N = 500 POIS)

k	Garble	OT	GC Eval	Total (s)
1	12,857.3 ± 684.4	<2.5	8,944.5 ± 1,531.2	24.5 ± 2.0
3	12,367.6 ± 470.6	<2.5	7,571.3 ± 1,022.1	21.9 ± 2.1
5	11,678.7 ± 574.5	<2.5	8,112.3 ± 1,342.1	23.7 ± 3.6
10	15,136.6 ± 3,046.0	<2.5	8,745.9 ± 1,481.4	27.4 ± 6.0

All times in ms except Total (s). n = 500 POIs; 3–5 trials per k.

B. Circuit Metrics, Candidate Scaling, and Tampering

Measured circuit metrics for $n = 500$, $b = 32$ (Section V-B): 1,122,000 AND gates, 2,244,000 AES calls, ≈35.06 MB garbled circuit, and a 250 KB simulated-OT payload—identical across all tested k, an open question we do not resolve here. Every recorded trial used the full $n = 500$ candidate set, since the demo deployment's coarse-discovery step never returned a smaller bound in the region tested; no candidate-size scaling curve at $n < 500$ was obtained. Across the recorded runs, every genuine result verified successfully against the Merkle root; however, no result was deliberately corrupted, so rejection of tampered results remains an untested property of the current evaluation.

X. CONCLUSION AND FUTURE WORK

We described VEIL as it is actually implemented: a two-party client/backend architecture combining disclosed coarse candidate discovery, a garbled-circuit distance/ranking computation that keeps exact coordinates client-side, and Merkle-verified results—with no trusted authority, fog-server tier, identity-based encryption, or cryptographically secure OT in the current prototype. Measured against a live 500-POI dataset, correctness matched a plaintext reference exactly across tested k, and every genuine result verified; end-to-end latency, however, measured approximately 20–27 seconds per query, far from real-time. We report these figures, and the gaps that remain (Section VIII), candidly rather than aspirationally.

Future work: (i) integrate a standard cryptographic OT protocol (e.g., Naor–Pinkas [9] with IKNP [8]) to replace the simulated mechanism; (ii) extend the security analysis toward the malicious-adversary model [10]; (iii) evaluate a geodesic distance computation as an alternative to the scaled squared-Euclidean approximation; (iv) substantially improve latency via a lower-level or native/WebAssembly implementation; (v) run the candidate-size-scaling and Merkle-tampering experiments left open in Section IX; and (vi) reduce coarse-location exposure during candidate discovery.

REFERENCES

[1] S. N. Eshun and P. Palmieri, "A Cryptographic Protocol for Efficient Mutual Location Privacy Through Outsourcing in Indoor Wi-Fi Localization," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4086–4099, 2024.

[2] M. Gruteser and D. Grunwald, "Anonymous Usage of Location-Based Services Through Spatial and Temporal Cloaking," in *Proceedings of the 1st International Conference on Mobile Systems, Applications and Services (MobiSys)*, 2003, pp. 31–42.

[3] B. Palanisamy and L. Liu, "MobiMix: Protecting Location Privacy with Mix-Zones over Road Networks," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2011, pp. 494–505.

[4] Z. Chen, J. Nie, Z. Li, W. Susilo, and C. Ge, "Geometric Searchable Encryption for Privacy-Preserving Location-Based Services," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2672–2685, 2023.

[5] H.-J. Kim, H.-I. Kim, and J.-W. Chang, "A Privacy-Preserving kNN Classification Algorithm Using Yao's Garbled Circuit on Cloud Computing," in *Proceedings of the IEEE International Conference on Cloud Computing*, 2017, pp. 766–769.

[6] V. Kolesnikov and T. Schneider, "Improved Garbled Circuit: Free XOR Gates and Applications," in *Proceedings of the International Colloquium on Automata, Languages and Programming (ICALP)*, 2008, pp. 486–498.

[7] S. Zahur, M. Rosulek, and D. Evans, "Two Halves Make a Whole: Reducing Data Transfer in Garbled Circuits Using Half Gates," in *Advances in Cryptology — EUROCRYPT 2015*, Lecture Notes in Computer Science, vol. 9057, Springer, 2015, pp. 220–250.

[8] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank, "Extending Oblivious Transfers Efficiently," in *Advances in Cryptology — CRYPTO 2003*, Lecture Notes in Computer Science, vol. 2729, Springer, 2003, pp. 145–161.

[9] M. Naor and B. Pinkas, "Oblivious Transfer and Polynomial Evaluation," in *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing (STOC)*, 1999, pp. 245–254.

[10] Y. Lindell and B. Pinkas, "Secure Two-Party Computation via Cut-and-Choose Oblivious Transfer," *Journal of Cryptology*, vol. 25, no. 4, pp. 680–722, 2012.

[11] A. C.-C. Yao, "How to Generate and Exchange Secrets," in *Proceedings of the 27th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1986, pp. 162–167.